

네트워크 구조

네트워크 구조 개요

▶ 네트워크 구조의 종류

여러 네트워크를 지속 유지하라!

- ▶ 이제 우리가 네트워크에서 병렬처리로 넘어온 근본적 원인을 다뤄보려 합니다.
- ▶ 결국 우리는 '상대방의 입력'과 '출력'을 동시에 받고 싶어한다는 것입니다.
- ▶ 그렇게 하기 위해서는 항상 'recv'를 수행하는 코드와, 'send'를 수행하는 코드가 멈추지 않아야합니다.
- ▶ recv의 경우 항상 데이터를 받기 위해 멈추기 때문에 여기서 블로킹이 되어버립니다.
- ▶ 이는 계속 send도 해야하는 프로그램이라면 문제로 작동합니다.
- ▶ 그렇기 때문에 우리는 이를 병렬처리해야 됩니다.
- ▶ 이제 그럼 문제는 '어떻게 recv와 send를 동시에 작동하게 만들 것인가?'입니다.
- ▶ 이를 위해 네트워크 구조에 대한 간단한 설명을 하도록 하겠습니다.
- ▶ 공식적으로는 Network Architecture 혹은 Network Model이라고 부르는 구분입니다.

Dedicated Server

- ▶ '내 모든 처리는 서버가 도맡아서 하겠다! 그대들은 오직 즐기기만 하라!'
- ▶ 바로 가장 강력한 형태의 서버입니다.
- ▶ 모든 신호는 서버로 향하고, 서버가 이를 처리하여 모든 클라이언트에게 뿌려줍니다.
(물론 이를 위해 방 단위로 묶을 지, 세션 단위로 묶을 지 등은 구조에 따라 다름)
- ▶ 다만 이렇게 서버가 '부담'을 지기 때문에 서버 비용부터 요구되는 서버의 컴퓨터 스펙이 매우 높습니다.
- ▶ 그러나 이걸 고수하는 이유는 단순합니다.(공짜 점심 없음)
- ▶ '신호 조작을 서버가 확인할 수 있다.'
- ▶ 알다시피 프로세스들은 결국 메모리가 정한대로 작동하게 되어 있습니다.
- ▶ 그럼 우리는 이 메모리를 '조작'할 수도 있다는 의미를 가집니다.
- ▶ 그러나 서버가 이 메모리들을 일일이 받아서 '어? 이런 수치는 비정상적인데?'라고 판정한다면 불안정한 클라이언트를 배제할 수가 있습니다.
- ▶ 더불어 핑 문제도 '공평'하게 만들어줍니다.

Dedicated Server

- ▶ 대부분 서버들은 국가나, 지역별로 하나를 위치시킵니다.
- ▶ 이유는 간단합니다.
- ▶ 결국 물리적 거리에 따른 반응속도의 차이가 발생하기 때문입니다.
- ▶ FPS 게임을 예를 들시다.
- ▶ FPS 게임 같은 경우에는 TTK(Time To Kill)이 매우 짧기로 소문난 종류입니다.
- ▶ 적과의 교전은, 단순 1초로 계산하기도 많을 만큼, 짧은 교전을 가집니다.
- ▶ 이런 짧은 교전 속에서 누구는 ping이 0.5초, 누구는 ping이 0.1초라면 어떨까요?
- ▶ 상대가 날 발견하고 방아쇠를 당기는 순간, 본인은 이제야 겨우 적을 발견 했을지도 모를 겁니다.
- ▶ 거기다 누군가가 방장을 하면, 그 스스로는 항상 ping이 0일 것입니다.
- ▶ 이건 '공평'하지 않습니다.
- ▶ 때문에 FPS와 같은 게임류는 중앙 서버가 이를 모두 관리하고 조정합니다.
- ▶ 한국 평균이라면 크게 플레이어마다 ping차이는 심하지 않아 '대체로' 공평할겁니다.

Dedicated Server

- ▶ 다만 일부 Dedicated Server는 중앙서버를 '위임'하는 경우가 있습니다.
- ▶ 즉, '플레이어야, 니가 서버해'라고 시키는 겁니다.
- ▶ '마인크래프트'나 '러스트', '테라리아'와 같은 중앙에서 관리하기에는 부담이 너무 큰 게임들, 그러나 소수로 운영한다면 충분히 현존 컴퓨터가 부담 가능한 형태의 서버를 플레이어에게 위임합니다.
- ▶ 이런 게임들류는 덕분에 쉽게 '모드(Mod)' 개발이 이뤄집니다.
- ▶ 왜냐면 클라이언트단과 서버단 모두를 플레이어가 가지고 있으니, 이를 비교 확인해 마음대로 조작할 수 있기 때문입니다.(철저히 막는 경우도 있지만 서도)
- ▶ 이런 서버의 경우는 이 게임이 '전세계 플레이어들이 서로 경쟁' 하지 않을 때 사용합니다.
- ▶ 그러나 FPS 게임 같은 경우에는 이렇게 운영하지 않습니다.(서버비 부담 높아짐)
- ▶ 때문에 거대 FPS 게임들이 상대적으로 BM(Business Model)이 매콤한 편입니다.
- ▶ 게임 개발은 물론 서버비도 부담되기 때문입니다.
- ▶ 서로 장단점이 있긴 합니다.

Listen Server

- ▶ 중앙에서 관리하는 서버가 있는가 하면, 철저히 '우린 몰라, 니네 알아서 해'라고 설계된 서버인 Listen Server도 존재합니다.
- ▶ 이 서버는 단순 '누구랑 누구랑 만나고 싶어한다고? 내가 매칭해줄게. 근데 그 다음부터는 니네 알아서 네트워크 관리해'라고 하고 서버를 떠넘깁니다.
- ▶ 대부분의 경우에는 서버 구현 코드가 클라이언트단에 포함되어 있고, 중앙 서버는 이를 클라이언트단에서 구성하고 '지시'만 내립니다.
- ▶ 이런 서버는 단순 '누구와 누구가 만나고 있고, 누구와 누가 현재 서버에 들어와 있다.'만 유지하면 되기 때문에 매우 자원소모가 적습니다.
- ▶ 그런 만큼 당연히 잃는 것도 있습니다.
- ▶ 당연히 이런 서버에서 '데이터 조작'은 심심치 않게 일어날 것입니다.
- ▶ 물론 이런 것에 대응이 아주 없는 것은 아니지만, 결국 이런 방식의 서버 운영은 모든 문제를 해결할 수는 없을 것입니다.
- ▶ 그래서 서버가 선택한 방법은 바로, '그냥 그런 문제가 생길 것 같은 서버에는 쓰지 말자'입니다.

Listen Server

- ▶ Listen Server는 너무 복잡하고 공격의 영향을 받을 수 있을 만한 문제에는 애초에 활용하지 않습니다.
- ▶ 단순 카트라이더, 크레이지아케이드, 서든어택과 같은 대기방을 사용하는 프로그램들이 이 Listen Server를 이용합니다.
(물론 게임 실행 이후에는 P2P 등을 활용)
- ▶ 다만, 서버를 넘긴다는 것은, 앞선 Dedicated Server를 넘기는 것과는 사뭇 다릅니다.
- ▶ 로직 자체가, '서버에서 통제한다'라는 개념과 다르게 Listen Server는 위임을 하더라도 결국 중앙서버가 이를 항상 확인 통제합니다.
- ▶ 결국 Dedicated Server로 로비를 관리하고, 방 단위는 Listen Server를 이용, 다시 방 단위 내부에서는 위임된 결과를 통해 P2P를 이용하여 서버를 구조화한 것입니다.
- ▶ 이런 여러 네트워크 구조를 겹치는 이유는 단순합니다.
- ▶ 각 서버가 맡은 일 자체가 다르기 때문입니다.
- ▶ 그럼 이런 서버를 합친 것을 알아보기 전에 P2P에 대해서도 알아보겠습니다.

P2P(Peer to Peer)

- ▶ '나로부터 너에게'라는 의미를 가지고 있습니다.
- ▶ 'Peer(피어)'자체가 '동료'라는 의미를 가지고 있습니다.
- ▶ P2P 서버는 모두가 '동등한 서버'의 역할과, '동등한 클라이언트'의 역할을 수행합니다.
- ▶ 특히 이 기술은 '파일 공유'의 혁신이라 불렸던 '푸루나', '당나귀', '토렌트' 등에도 사용된 기술입니다.
- ▶ '1:다'의 Dedicated Server와 다르게 '다:다'의 형태를 띄고 있는 P2P는 서로가 서버이자 서로가 클라이언트인 구조입니다.
- ▶ 이게 왜 '푸루나'나 '당나귀', '토렌트'와 같은 파일 공유 프로그램의 핵심일까요?
- ▶ 바로 '모두가 이 파일 전송을 위해 조금씩 모두에게 보내주고 있다.'가 핵심입니다.
- ▶ P2P 프로그램을 쓰면 컴퓨터 느려진다는 말이 괜히 생긴 말이 아닙니다.
- ▶ 서로가 서버이다보니 서로에게 데이터를 서로에게 각각 보내고 있는 것입니다.
- ▶ 그게 P2P의 핵심이기 때문입니다.

P2P(Peer to Peer)

- ▶ 여러분은 아마 P2P 파일 공유 프로그램으로 무언가를 다운로드 받을 때 '속도가 잘 유지된다!'라는 식의 발언을 들어보았을 것입니다.
- ▶ 그 이유는 여러 명이 하나의 파일을 조각내서 조금씩의 크기로 뿌려주기 때문입니다.
- ▶ 단순 한 사람은 겨우 1KB의 속도로 데이터를 전송할지 모르지만, 받는 측은 수많은 사람들에게 받은 데이터 속도로 500KB속도로 다운로드가 되는 그런 원리입니다.
- ▶ 이처럼, P2P 파일 공유 프로그램의 핵심은 각자가 서버 역할도 동시에 한다는 것입니다.
- ▶ 특히, '아프리카', '치지직'과 같은 동영상 플랫폼은, 수 많은 사람들에게 스트리밍하기 위해서 자신의 서버를 이용하기 보다는, 각 스트리밍을 보는 사람들의 자원을 끌어다 서버의 부하를 줄이는 P2P 기술을 적절히 이용하고 있습니다.(이를 위해 전용 프로그램을 깔라고 하고, 사용을 유도하기 위해 고화질 영상 제공을 미끼로 쓴다!)
- ▶ 특히 이런 P2P연결은 Ping이 낮은 사람들끼리는 낮게, 높은 사람은 Ping이 높으면 매칭해주지 않거나, 우선순위를 낮춰 매칭하므로 반응속도가 중요한 게임에서도 자주 사용됩니다.

Hybrid Architecture

- ▶ 아마 소프트웨어단 네트워크의 난이도의 가장 큰 역할을 하는 부분일 것입니다.
- ▶ 바로 단순 Dedicated Sever, Listen Server, P2P 서버를 구분하지 않고 모두 섞어 필요한 부분에 배치하는 네트워크 구조입니다.
- ▶ 중앙집권형의 Dedicated Server나, 분산형의 P2P, Listen이냐를 떠나서 전부 섞어서 필요한 곳에 배치시키는 것입니다.
- ▶ 당연히 여러 로직과 복잡한 알고리즘, 여러 구조를 하나의 로직으로 다루는 방식 때문에 구현자체는 '매우 어렵습니다.'
- ▶ 물론 현대에 와서 어느정도 정형화된 서버의 형태가 있기 때문에 이를 위한 라이브러리가 존재하긴 하지만, 결국 어려운건 매한가지입니다.
- ▶ 결국 어떤 구조가 자신의 프로그램에 어울리고 어떻게 설계할 것인가, 그리고 어떠한 프로토콜을 이용하여 통신할 것인가는 전적으로 개발자의 선택에 달렸습니다.

저수준? 고수준?

- ▶ 이쯤 되면 '그럼 그냥 라이브러리 사용해서 만들면 되는거 아닌가? 굳이 소켓통신을 내가 만드는 이유가 뭐지?'라는 고민도 할 것입니다.
- ▶ 그러나 이 모든 라이브러리들의 기본은 '소켓 단위 통신'입니다.
- ▶ 그렇기 때문에 '고수준 동작'을 이해하기 위해서는 항상 '저수준 동작'이 머리에 있어야 합니다.
- ▶ 이 저수준 동작을 넘어서는 동작은 할 수 없기에 고수준도 저수준의 기준에서 생각해야 한다는 것입니다.
- ▶ 단순 라이브러리를 쓴다는 것에 매몰된다면 디버깅도 힘들어집니다.
- ▶ 라이브러리가 아닌 소켓 자체 단의 문제인데도 라이브러리의 문제로 찾는 경우도 더러 있기 때문에 항상 '저수준'의 이해가 수반되어야 합니다.
- ▶ 그러나 저수준을 너무 깊게 들여다보는 것도 추천되지 않습니다.(심연을 쳐다본다면 심연도 당신을 쳐다볼 것이다.)
- ▶ 자신이 네트워크를 다룸에 있어서 어느정도 선이 적절한지 스스로 찾아가는 습관이 필요합니다.

네트워크 구조 구현

네트워크 구조 구현 개요

- ▶ 멀티 쓰레딩 기반 채팅 프로그램 구현
- ▶ 멀티 플렉싱 기반 채팅 프로그램 구현

멀티 쓰레딩 기반 1:1 채팅 구현

- ▶ 이제 이 병렬처리를 이용해서 동시에 입력과 출력을 받을 수 있는 서버와 클라이언트를 구현해보겠습니다.
- ▶ 전체적인 구조의 흐름은 다음과 같습니다.
- ▶ 서버단
 - '서버 구동' -> '소켓 생성 및 클라이언트 대기' -> '클라이언트 접속 시 send만하는 thread와 recv만하는 thread를 생성 및 구동' -> '통신이 끊기면 프로그램이 종료됨'
- ▶ 클라이언트단
 - '클라이언트 구동' -> '소켓 생성 및 서버에 접근' -> '서버에 접속 시 send만 하는 thread와 recv만 하는 thread를 생성 및 구동' -> '통신이 끊기면 프로그램이 종료됨'
- ▶ 이 구조는, 오직 1:1로 작동하고 구조상으로 '방단위 통제'라던가 '여러명이 한 채팅방에'와 같은 구조는 작동하지 않습니다.
- ▶ 즉, 그런 구조를 원한다면 실제 그런 구조 구현을 해야합니다.
- ▶ 그리고 입력중에 출력이 일어나면 글자가 찢리는 현상도 제한적으로 터미널의 한계로서 가지고 있습니다.
- ▶ (코드는 카페에)

멀티플렉싱 기반 다:다 채팅 프로그램

- ▶ 이번엔 서버는 오직 '채팅 전달'만 신경쓰고, 클라이언트간에만 작동하는 채팅 프로그램을 만들어보겠습니다.
- ▶ 전체적인 구조의 흐름은 다음과 같습니다.
- ▶ 서버단
 - '서버 구동' -> '소켓 생성' -> '멀티 플렉싱 구현' -> '멀티 플렉싱 소켓별 작동 방식 구현'(서버 소켓은 클라이언트 접속, 각 클라이언트 소켓은 넘어온 채팅 데이터'
- ▶ 클라이언트단
 - '클라이언트 구동' -> 소켓 생성 및 서버에 접근' -> '서버에 접속 시 send만 하는 thread와 recv만 하는 thread를 생성 및 구동' -> '통신이 끊기면 프로그램이 종료됨'
- ▶ 이 구조는 중앙의 서버가 모든 클라이언트들의 대화를 다시금 자신에게 붙은 모든 클라이언트들에게 다시 뿌려주는 방식을 사용함으로써, 굳이 P2P 방식의 클라이언트가 다른 클라이언트들을 모두 알필요 없게 합니다.
- ▶ 서버는 어차피 자신에게 붙을 것이니 알기 쉽고, 통제하기가 편합니다.
- ▶ 그리고 이 전제에는 '채팅 정도는 서버가 감당가능하다'가 깔려 있어야 합니다.
- ▶ (코드는 카페에)

전용 프로토콜

전용 프로토콜 개요

- ▶ 전용 프로토콜이란?
- ▶ 전용 프로토콜의 필요성
- ▶ 전용 프로토콜 구조화

전용 프로토콜이란?

- ▶ 전용 프로토콜을 다르게 부르면 '커스텀 프로토콜'입니다.
- ▶ 즉, 개개인이나 회사가 국제 규격 이외에 자신의 통신에 활용하기 위해서 만든 프로토콜을 의미합니다.
- ▶ 그런데 이미 국제 규격의 프로토콜이 이것저것 있는 시대에 왜 이런 전용 프로토콜을 만들어야 할까요?
- ▶ 그 이유는 간단합니다.
- ▶ 이 세상에 모든 프로그램에 활용할 일반적인 프로토콜을 만들 수도 없으며, 이런 모든 프로그램에 사용할 프로토콜들을 국제 규격으로 하기도 어렵기 때문입니다.
- ▶ 회사나 개인은 저마다 자신이 원하는 형태의 서비스를 만들어 제공할 텐데, 국제 단위에서 이 모든 프로그램에 맞는 프로토콜을 관리하는 것은 손해이며, 모두에게 혼란을 가중할 뿐입니다.
- ▶ 때문에 국제가 맞춰야 할 프로토콜만 국제 규격으로 만들어 들 뿐입니다.
- ▶ 뭐 간단히 설명하면 USB 자체는 국제 규격이지만, USB 디자인은 회사마다 다르다는 것으로 이를 알 수 있을 것입니다.

전용 프로토콜이 왜 필요한가?

- ▶ 그럼 왜 전용 프로토콜이 필요할까요?
- ▶ 이전 멀티 플렉싱 채팅을 떠올려 보겠습니다.
- ▶ 정상 종료는 상대방이 종료 메시지를 전달 받을 수 있습니다.
- ▶ 그런데 비정상적으로 종료한다면 어떨까요?
- ▶ 이 경우에는 '종료 메시지'를 전달 받지 못합니다.
- ▶ 왜냐면 그동안 항상 '종료 메시지'는 나가려고 시도한 클라이언트가 보내주었기 때문입니다.
- ▶ 그럼 서버는 클라이언트 종료를 `str_len`을 반환된 값을 통해서 알 수 있지만, 나머지는 알 수 없습니다.
- ▶ 때문에 서버가 이들에 대한 정보를 미리 받아놓아야 하고 그 데이터에 맞게 비정상 종료에 대해서도 모든 사람들에게 이 메시지를 보내주어야 합니다.
- ▶ 거기다, 유저 정보를 알면 '중복 ID'를 서버가 사전에 차단하고, 때에 따라선 다시 클라이언트에게 '그런 아이디는 안된다'라는 신호도 보낼 수 있습니다.
- ▶ 그럼 어떻게 보내야 할까요?

구조화된 데이터, 그러나...

- ▶ 당연히 배열이나 구조체와 같이 구조화 되어 있는 구조를 이용하면 좋겠지만...
- ▶ 통신에는 그저 '바이트 스트림' 혹은 '텍스트'를 보낼 수 있을 뿐입니다.
- ▶ 물론 바이트 스트림단위로 구조체를 직접 데이터로 보내는 방법도 있지만, 구조체의 단점은 '크기가 정해져 있다.'라는 것입니다.
- ▶ 뭐 간단한 내용입니다.
- ▶ 지금 보면 알겠지만 채팅 로직을 보면 항상 1024로 보내고 1024로 받고 있습니다.
- ▶ 만약 1024를 넘는 글자를 쓰거나 부족하게 썼을 때는 어떨까요?
- ▶ 넘는 글자는 오류를 불러올 수 있고, 부족할 경우에는 나머지 바이트는 낭비될 것입니다.
- ▶ 그럼 우리가 원하는 건 하나입니다.
- ▶ 매번 크기에 맞게 데이터를 보내는 방법은?
- ▶ 그리고 알다시피 배열이나 구조체는 크기가 고정일텐데 당연히 이건 안되겠죠?
- ▶ 그럼 현업에서는 어떤 방식으로 구조화된 데이터를 주고 받을까요?

선 헤더 전송 / 후 데이터 전송

- ▶ 헤더(Header)를 먼저 보내는 것입니다.
- ▶ 이 헤더는 항상 고정 크기일 겁니다.
- ▶ 그 대신 이 헤더는 다음으로 받을 데이터에 대한 정보를 포함하고 있습니다.
- ▶ 우리는 이 헤더를 해석함으로써 데이터가 어느 정도 크기로 오는지 어떤 형태로 해석해야하는지에 대한 정보를 미리 받을 수 있는 것입니다.
- ▶ 그렇게 되면 그 정보를 토대로 데이터를 전송 받을 수 있는 것입니다.
- ▶ 이렇게 한다면 통신을 2번 한다는 단점은 있지만, 그에 비해 얻을 수 있는 이점이 더 많습니다.
- ▶ 특히 '대용량 데이터'를 주고 받을 때 더욱 빛을 발합니다.
- ▶ 게다가 이 경우에는 1바이트씩 데이터가 차이가 나더라도 헤더를 통해서 가변크기를 알려주기 때문에 모든 데이터 크기에 대응시킬 수 있습니다.
- ▶ 그리고 이런 헤더를 만드는 것은 일련의 '프로토콜'을 만드는 것입니다.
- ▶ 그럼 이런 선 헤더 전송 기법을 통해 어떤 구조화된 데이터를 전송할 수 있을까요?

JSON(JavaScript Object Notation)

- ▶ 자바스크립트가 좋은 언어냐 나쁜 언어냐는 논쟁을 떠나서 JSON은 현대 통신의 주축에 서있는 일종의 '전용 프로토콜 생성용 언어'입니다.
- ▶ JSON은 데이터 포맷으로서 대괄호, 중괄호 등을 이용하여 데이터를 구분, 배열처럼 다룰 수 있으면서, 자료형들처럼 다루듯이 하여 문자면 문자, 숫자면 숫자 따로 보낼 수 있으며, 이 모든 형태는 키-값쌍 형태로 되어 있습니다!(마치 딕셔너리처럼!)
- ▶ 그리고 JSON은 Data Serializtion Format(데이터 직렬화 포맷)으로 불리는 '단일 스트림으로 표현할 수 있는 구조화된 데이터'이기에 매우 통신에 유리합니다.
- ▶ 즉, 구조화된 데이터이지만 하나의 바이트 스트림으로 만들 수 있어, 통신으로 그대로 전송하더라도 구조가 무너지지 않는다는 것입니다.
- ▶ 현존 JSON을 해석(Parsing, 파싱)하는 여러 라이브러리들이 존재하기에 쉽게 자신의 통신 서버에 적용할 수도 있습니다.
- ▶ 다만 JSON은 이 파싱 과정 때문에 일부 대용량 처리를 해야하는 게임 서버에게는 부담이지만, 일반적으로 서버들이 상시 대용량을 처리하지 않기에, 매우 쉽게 구조화된 데이터 전송에 활용될 수 있습니다.

네트워크 기본을 마치며...

- ▶ 네트워크는 결국 얼마만큼 기본기로 구조화 시키느냐의 문제입니다.
- ▶ 거기에 병렬처리가 가미되며, 여러 전용 프로토콜이 포함되어야 합니다.
- ▶ 기본기가 안된다면 아마 앞으로의 모든 통신도 이해하기 힘들 것입니다.
- ▶ 그러나 이 모든 기본적 통신을 기반으로 통신들이 만들어진다는 사실은 국제 규격이 바뀌지 않는 이상은 항상 같기에 너무 고급 기술에 허덕이지 않아도 됩니다.
- ▶ 설령 압니까?
- ▶ 여러분들이 기본기로 만든 프로토콜이 국제 규격이 될지?
- ▶ 이를 통해서 여러 네트워크 프로젝트를 시작해보겠습니다!